Аккредитованное образовательное частное учреждение
высшего образования
«Московский финансово-юридический университет МФЮА»
(МФЮА)

КАФЕДРА «ИНФОРМАЦИОННЫЕ СИСТЕМЫ И ТЕХНОЛОГИИ»

КУРСОВАЯ РАБОТА

по дисциплине: «Разработка программных модулей»

на тему: «Разработка модуля управления каталогом товаров для интернет-магазина»

Специальность: 09.02.07
Информационные системы и программирование
Профиль: Программист

Автор работы: Чемоданов Никита Сергеевич
Курс: 4 Группа: 23ИПо8381 (ИПо-41)
Руководитель работы: Колпащиков Кирилл Александрович
Подписи и даты: _____

Киров — 2026

СОДЕРЖАНИЕ

Введение.....3
1 Теоретические основы и анализ управления товарным каталогом.....5
1.1 Особенности ведения каталога товаров интернет-магазина.....5
1.2 Подходы к автоматизации и требования к модулю.....6
1.3 Обоснование выбора технологий.....9
2 Проектирование модуля управления каталогом.....12
2.1 Пользовательские сценарии и функциональная модель.....12
2.2 Проектирование базы данных и ограничений целостности.....15
2.3 Архитектура приложения и проектирование интерфейса.....19
3 Разработка и тестирование модуля.....22
3.1 Реализация операций с товарами и категориями.....22
3.2 Реализация поиска, фильтрации и веб-интерфейса.....24
3.3 Тестирование, анализ результатов и ограничения решения.....27
Заключение.....32
Список использованных источников.....33
Приложение А. Экраны и результаты демонстрации.....36

ВВЕДЕНИЕ

Менеджер каталога вносит товары, меняет цены и остатки, распределяет позиции по категориям. При вводе возможны ошибки: повторный артикул, отрицательное количество или ссылка на несуществующую категорию. Разработанный модуль проверяет эти данные и сообщает о причинах отказа. Для работы с каталогом предусмотрены веб-страницы и JSON API.

Работа выполняется по дисциплине «Разработка программных модулей» для специальности 09.02.07 «Информационные системы и программирование». Выданное задание определяет операции добавления, просмотра, изменения и удаления товаров, поиск, фильтрацию, сортировку и хранение данных. В качестве документационной основы в задании назван ГОСТ 19.201–78 [1]. Структура и базовое оформление записки приняты по предоставленным методическим указаниям МФЮА [2]. Объём проекта ограничен управлением каталогом: заказы, расчёты с покупателями и доставка не рассматриваются.

При проектировании использованы материалы о типах и классах Python [3], разделении прикладных операций и обработчиков запросов [5], работе с небольшими воспроизводимыми примерами [7]. Для выбора проверок рассмотрены функциональные тесты пользовательских сценариев и тесты отдельных компонентов [6; 8]. В проекте эти виды проверок разделены на тесты базы данных, API и HTML-страниц.

Цель работы — разработать и проверить модуль управления каталогом товаров интернет-магазина. Для этого необходимо определить требования к товарам и категориям, выбрать средства разработки, спроектировать базу данных и интерфейс, реализовать операции каталога. Проверка результата включает подготовку демонстрационных данных, испытание допустимых и ошибочных запросов, а также описание ограничений программы.

Объект исследования — процесс ведения товарного каталога интернет-магазина. Предмет исследования — структуры данных, алгоритмы обработки запросов и программные средства автоматизации хранения, изменения и поиска

сведений о товарах и категориях. В качестве учебного ассортимента используются канцелярские товары. Сведения не относятся к обследованному предприятию; цены и остатки в демонстрационном наборе условные.

В работе использованы анализ документации, сравнение способов хранения, моделирование данных и функциональное тестирование. Для проверки операций сопоставляются ответ сервера и состояние базы: после допустимого запроса запись должна измениться, после отклонённого — сохранить прежние значения. Экономический эффект и время работы менеджера не измерялись.

Результат работы — приложение, исходный код, инструкция запуска и тесты. Первая глава содержит описание предметной области, требования и выбор технологий. Во второй главе рассматриваются модель данных и устройство приложения. Третья глава посвящена реализации и испытаниям. Снимки страниц приведены в приложении.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АНАЛИЗ УПРАВЛЕНИЯ ТОВАРНЫМ КАТАЛОГОМ

1.1 ОСОБЕННОСТИ ВЕДЕНИЯ КАТАЛОГА ТОВАРОВ ИНТЕРНЕТ-МАГАЗИНА

Каталог содержит записи о товарных позициях. В каждой указаны название, артикул, категория, цена, описание и количество. Название используется для отображения и поиска, артикул отличает одну позицию от другой. Например, ручки с одинаковым кратким названием могут иметь разные цвета и толщину линии. Для раздельного ведения таких позиций им назначаются разные артикулы.

Идентификатор записи создаётся базой и используется в связях и адресах карточек. Артикул задаёт менеджер. Если он исправляет артикул товара, идентификатор остаётся прежним: меняется существующая запись. Уникальность нового артикула проверяется среди остальных товаров.

Категория объединяет товары по назначению. Её название и описание хранятся в отдельной записи, на которую ссылаются товары. При переименовании меняется одна запись справочника, связи с товарами сохраняются. В проекте каждому товару назначается одна категория; вложенные категории не предусмотрены.

Цена задаётся в рублях с точностью до копейки и не может быть отрицательной. Остаток содержит целое число единиц, начиная с нуля. Товар с нулевым остатком остаётся в общем списке, но исключается из отбора по наличию. Карточка показывает текущее количество; поступления, списания и резервирование отдельными операциями не учитываются.

Для товара и категории в программе определены модели. Состав входящего запроса проверяется отдельной схемой, а для ответа подготавливаются нужные клиенту поля. В книге Think Python рассмотрены классы для описания пользовательских типов и словари для хранения пар ключ

–значение [3, гл. 10, 14]. В проекте класс модели содержит поля записи, а словарь значений формы сохраняет введённый текст при ошибке. Эти объекты используются на разных этапах обработки запроса.

Основной пользователь модуля — менеджер каталога. Он ведёт справочник категорий, добавляет и исправляет товары, ищет нужные позиции и удаляет ненужные записи. Результат действия отображается в карточке или списке. Работа не требует отдельного клиентского приложения.

Одна запись описывает товарную позицию, а не отдельную физическую единицу. Десять одинаковых ручек представлены одним товаром с остатком 10. Для синих и чёрных ручек могут потребоваться отдельные записи: у них разные артикулы, а остатки изменяются независимо. При этом категория и цена могут совпадать.

Для названий категорий действует запрет повторов без учёта регистра: «Бумага» и «БУМАГА» обозначают один раздел. Названия товаров могут совпадать, поскольку их уникальность определяется артикулом. Правила сравнения задаются отдельно для каждой сущности.

К обязательным сведениям о товаре относятся название, артикул, категория, цена и остаток. Описание дополняет карточку и может отсутствовать. Даты создания и изменения назначаются программой. Из этого состава данных определяются поля формы и ограничения при сохранении. Название категории ограничено 100 символами, название товара — 200. Для описаний допускается до 2000 символов. Пределы одинаковы в схеме запроса и моделях; они учитываются при подготовке HTML-полей. Артикул содержит до 64 латинских букв, цифр, дефисов или подчёркиваний.

1.2 ПОДХОДЫ К АВТОМАТИЗАЦИИ И ТРЕБОВАНИЯ К МОДУЛЮ

Для ведения каталога рассмотрены электронная таблица, готовая система магазина и отдельный программный модуль. Сравниваются хранение свойств товара, связь с категориями, проверка ввода и обработка запросов. Оценка

касается структуры решения; испытания коммерческих продуктов и расчёт стоимости внедрения не проводились.

В электронной таблице товары располагаются по строкам, а свойства — по столбцам. Проверки артикулов и связей с категориями нужно настраивать отдельно. Готовая система предоставляет собственную модель и интерфейс, которые требуется подобрать под задачу. Разработка модуля даёт возможность определить поля, операции и веб-страницы в соответствии с учебным заданием.

В проекте реализован отдельный веб-модуль на Python. В нём обработка запроса, проверка данных и запись в базу доступны для разбора по исходному коду. Сравнение подходов приведено в таблице 1. Построение небольших запускаемых примеров рассматривается также в учебных материалах Г. Уилсона [7].

Таблица 1 — Проектное сравнение подходов

Подход	**Что организуется**	**Соответствие задаче**
Таблица товаров	Столбцы свойств, правила ввода и связи нужно настроить	Подходит для исходного перечня; HTTP-модуль не реализует
Готовая система	Выбор продукта, настройка или расширение	Требует выбора продукта и настройки под задачу
Отдельный модуль	Собственные схема, операции, формы и проверки	Содержит операции каталога на Python и FastAPI

Бизнес-требования определяют назначение модуля: ведение каталога, обновление цен и остатков, поиск позиций и предотвращение недопустимых записей. Пользовательские требования описывают действия менеджера, функциональные — поведение системы. Например, для исправления цены система должна принять новое значение, проверить его и записать в выбранный

товар. Уникальность артикула и допустимые числовые значения задаются бизнес-правилами.

Функциональные требования включают создание, просмотр, изменение и удаление товаров и категорий, подробную карточку товара, поиск по названию, фильтрацию категории и цены, признак наличия, сортировку и постраничный вывод. Условия отбора должны сочетаться. Поиск не зависит от регистра русского текста; ценовые границы включают равные значения. Страница списка содержит не более десяти товаров, а ссылки переходов сохраняют выбранные условия. Для сортировки предусмотрены название и цена в обоих направлениях, а также время последнего изменения. Неизвестное значение сортировки должно отклоняться. Номер страницы задаётся положительным целым числом. Поиск ограничен 200 символами. В совокупности эти правила определяют допустимый запрос списка до обращения к базе.

К данным предъявляются требования целостности: существующая категория у каждого товара, уникальный артикул, неповторяющееся нормализованное имя категории. Цена и остаток должны быть неотрицательными, количество — целым. Категорию с товарами удалять нельзя. Зафиксированные записи должны сохраняться между запусками сервера.

Для каждой операции проверяется успешный запрос и причины отказа. При создании товара это повторный артикул, неправильная цена и отсутствующая категория; при редактировании дополнительно проверяется сохранность прежних данных после ошибки. Вводная глава The Fuzzing Book объясняет проверку через сопоставление результата с ожидаемым поведением и отмечает ограниченность отдельных тестовых примеров [10].

Выполнение требований проверяется на уровне запроса и хранения. Так, при повторном артикуле API должен вернуть 409, а таблица — отклонить повтор по ограничению UNIQUE. Проверка ответа без последующего чтения базы не показывает, сохранились ли прежние данные. Поэтому при испытаниях сравниваются оба результата.

Для приёмки операции задаются исходное состояние, действие и ожидаемый результат. Перед проверкой удаления занятой категории в базе должны существовать категория и связанный товар. Затем выполняется запрос удаления. Правильный результат включает отказ и сохранение обеих записей. Если проверить только текст сообщения, можно пропустить ошибочное изменение базы. По той же причине проверка редактирования должна читать запись после сохранения, а проверка неуспешного редактирования — сравнивать её с прежними значениями.

Требования к интерфейсу связаны с выполнением действий: поля должны иметь подписи, ошибка — понятное объяснение, удаление — отдельное подтверждение. При переходе между страницами должны сохраняться фильтры. Эти свойства проверяются по HTML-ответам и действиям в браузере. Исследование удобства с участием пользователей в работе не проводилось.

1.3 ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

Приложение написано на Python 3.12. Версии библиотек закреплены в файлах зависимостей, команды установки приведены в README. Для пакетов создаётся виртуальное окружение проекта. Сохранение сведений о среде и порядке запуска нужно для повторения проверок; этот вопрос рассмотрен в Research Software Engineering with Python [4].

FastAPI обрабатывает HTTP-запросы и предоставляет JSON API. Веб-страницы формируются с Jinja2: обработчик передаёт имя шаблона и данные для его заполнения. Подключение шаблонов и статических файлов описано в документации FastAPI [13]. Для запуска интерфейса достаточно сервера приложения и браузера; отдельная сборка клиентского кода не требуется. Шаблоны и статические файлы размещены в папках app/templates и app/static. Пути к ним вычисляются от каталога приложения. CSS и скрипт дат загружаются с того же сервера. Интерактивная документация API находится по адресу /docs; её файлы Swagger UI загружаются из CDN. Это отдельная страница, не требуемая для работы форм каталога.

Для работы с данными применяется SQLAlchemy, для входных схем — Pydantic. Сессия SQLAlchemy связывает загруженные объекты с транзакцией и предоставляет явные commit и rollback [14]. Строгий режим Pydantic ограничивает неявные преобразования типов [16]. В API количество и идентификатор категории принимаются целыми JSON-числами. HTML передаёт строки, поэтому адаптер формы отдельно разбирает допустимую целочисленную запись, затем применяет ту же схему.

В приложении используется SQLite: база размещается в локальном файле и не требует отдельного серверного процесса [11]. Для тестов можно создавать независимые файлы с такой же схемой. Документация PostgreSQL, указанная в задании, использована при рассмотрении таблиц и ограничений [12]. Действующая реализация работает с SQLite.

В приложении используются особенности SQLite: проверки typeof и GLOB, параметры подключения и команды PRAGMA. При переходе на PostgreSQL эти места придётся пересмотреть. Затем нужно заново проверить таблицы, ограничения и запросы на выбранной СУБД. В этой работе перенос не выполнялся. Тесты охватывают локальный запуск с SQLite. Допустимое число одновременно работающих менеджеров по этим результатам не установлено.

Тесты базы, API и HTML выполняются через pytest и тестовый HTTP-клиент, действия в браузере — через Playwright [17; 19]. Для локального запуска используется Uvicorn: ему передаются модуль приложения, адрес и порт [20].

Сервер возвращает готовые HTML-страницы. Для списка он подготавливает товары и параметры отбора, для формы — значения полей и перечень категорий. Основные действия выполняются обычными GET- и POST-запросами. JavaScript используется только для отображения дат в местном часовом поясе браузера.

В create_sqlite_engine задан параметр check_same_thread=False для соединений SQLite. При открытии соединения обработчик выполняет PRAGMA foreign_keys=ON, чтобы ограничения связей действовали для

запросов приложения. Каждая операция использует сессию, полученную обработчиком запроса; общая изменяемая сессия для всех запросов не создаётся. Проверка внешних ключей включается обработчиком события connect при каждом новом подключении. Поэтому она действует и при открытии файла через созданный приложением движок, и при создании временного движка в тестах. Функция init_db принимает движок параметром, что позволяет использовать одну схему для разных файлов.

Версии библиотек указаны в requirements.txt и requirements-dev.txt. После установки команда pip check проверяет совместимость зависимостей установленных пакетов. Далее запускается pytest, который выполняет запросы и проверяет данные. Для повторения демонстрации нужно получить исходники, создать окружение Python 3.12 и выполнить команды из README. При запуске приложение создаёт файл базы и недостающие таблицы. Сам рабочий файл базы в Git не хранится; после получения проекта его можно заполнить загрузчиком примеров.

2 ПРОЕКТИРОВАНИЕ МОДУЛЯ УПРАВЛЕНИЯ КАТАЛОГОМ

2.1 ПОЛЬЗОВАТЕЛЬСКИЕ СЦЕНАРИИ И ФУНКЦИОНАЛЬНАЯ МОДЕЛЬ

Первый шаг работы — создание категории с названием и необязательным описанием. После сохранения она появляется в списке выбора формы товара. При пустом справочнике интерфейс предлагает сначала создать категорию. Совпадающее нормализованное имя отклоняется.

Для нового товара менеджер вводит название, артикул, категорию, цену и остаток; описание необязательно. После успешного сохранения открывается карточка. Форма редактирования заполняется текущими значениями, а изменение относится к той же записи. Ошибочный запрос не должен менять её поля.

Менеджер может одновременно задать поиск, категорию, границы цены, наличие и сортировку. Условие, которое не выбрано, не ограничивает выдачу. Для пустого каталога предлагается создать первую запись, для пустого результата поиска — изменить условия. Связь требований с проверками приведена в таблице 2.

Таблица 2 — Связь требований с проверками

Требование	**Реализация**	**Признак корректности**
CRUD товара	Форма, карточка, API	Изменения видны после чтения; ID сохраняется
Категории	Справочник и связь	Повторное имя и удаление занятой категории отклоняются
Поиск	Нормализованное имя	Русский регистр не меняет результат
Фильтры	Условия общего запроса	Категория, цена и наличие применяются вместе
Пагинация	LIMIT 10 и OFFSET	120 страниц для 1200 товаров
Неправильный ввод	Схемы и форма	Статус 422; заполненные значения сохранены
Данные после старта	Файловая SQLite	Повторное открытие не удаляет записи

Удаление товара начинается с просмотра страницы подтверждения. Запрос GET показывает сведения, а POST выполняет удаление; отмена возвращает карточку. Для занятой категории выводится причина запрета и ссылка на её товары. Перед изменением базы сервер повторно проверяет наличие объекта и связанные ограничения.

API и страницы используют одинаковые коды основных ошибок: 422 для неверного ввода, 404 для отсутствующей записи и 409 для конфликта. Содержание ответа зависит от интерфейса: JSON или HTML. При создании через API возвращается 201, при удалении — 204 без тела. Сохранение веб-формы завершается перенаправлением 303.

Для каждого сценария определяется проверяемый результат. В книге Х. Персиваля функциональный тест описывает поведение программы с точки зрения пользователя; проверки отдельных частей помогают найти место ошибки [6, гл. 4]. В проекте запросы к API и HTML используются для проверки

действий с каталогом. Тесты базы проверяют ограничения таблиц. Браузерные сценарии воспроизводят работу с формами и переходы по ссылкам. Для каждого вида проверки сохранены команды запуска или отчёт.

При редактировании различаются три состояния: запись в базе, значения отправленной формы и страница результата. До фиксации транзакции новый ввод не считается сохранённым. Если запрос отклонён, менеджер получает форму для исправления. При успехе браузер открывает карточку с данными из базы.

Адрес карточки и подтверждение удаления содержат идентификатор записи. Положение товара в отсортированном списке для этих действий не используется. Если после открытия страницы товар удалён другим запросом, повторное получение по идентификатору завершается ответом 404.

Допустимый по формату идентификатор без соответствующей записи приводит к 404. Отрицательный или выходящий за числовой предел идентификатор отклоняется с кодом 422. Для поиска также различаются пустой результат при правильных условиях и ошибочный диапазон, в котором нижняя цена больше верхней. Отбор по несуществующей категории приводит к 404: перед составлением списка функция получает категорию по идентификатору. При существующей категории без подходящих товаров возвращается обычная пустая выдача. Таким образом различаются отсутствие объекта справочника и отсутствие результатов внутри выбранного раздела.

Ответ API после создания товара содержит идентификатор, название категории и цену строкой рублей. В тестах полученный идентификатор используется для последующего чтения, изменения и удаления. Это проверяет последовательную работу операций с одной записью, включая сохранность значений между запросами.

2.2 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ И ОГРАНИЧЕНИЙ ЦЕЛОСТНОСТИ

Модель состоит из категорий и товаров со связью один ко многим. Категория содержит первичный ключ id, название, его нормализованную копию и описание. У товара есть собственный id, артикул, category_id и поля характеристик. Состав полей и связь показаны на рисунке 1 и в таблице 3.

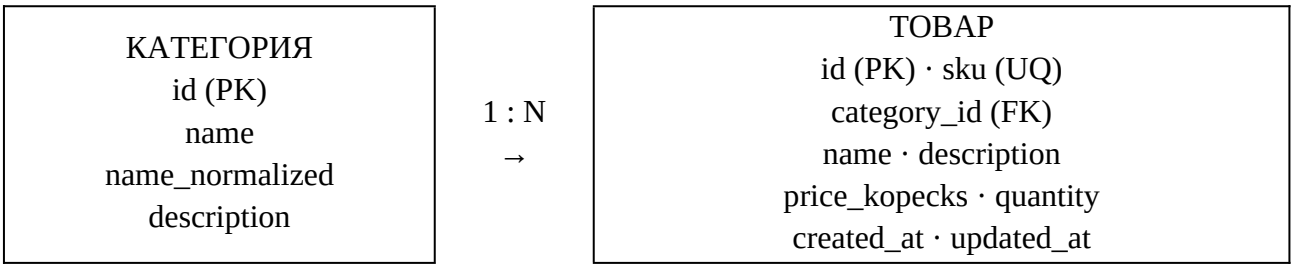


Рисунок 1 — Логическая структура данных каталога

Источник: составлено по модели и исходному коду проекта.

Категория хранится в таблице categories, товары ссылаются на неё через category_id. Для связи задан RESTRICT: при наличии связанных товаров удаление категории отклоняется самой базой. Принципы первичных и внешних ключей описаны в документации PostgreSQL [12]; в проекте они применены в таблицах SQLite через SQLAlchemy.

Цена хранится целым числом копеек. Строка рублей проверяется по допустимому формату и преобразуется через Decimal. Принимаются точка и запятая, не более двух знаков после разделителя; лишняя точность отклоняется без скрытого округления. Документация decimal объясняет возможность точного представления десятичных значений в отличие от ряда значений float [11]. При выводе целые копейки снова преобразуются в строку рублей с двумя дробными знаками.

Количество хранится целым числом в диапазоне SQLite. Входная схема отклоняет дробные и логические значения. В таблице ограничение CHECK проверяет тип и неотрицательность. Для артикула и нормализованного имени категории заданы уникальные индексы. Перед записью приложение ищет совпадение и может сразу сообщить, какой артикул занят. Ограничение базы

остаётся действующим при записи, в том числе если совпадающие запросы поступили почти одновременно.

Для поиска используется дополнительное нормализованное имя: NFKC, удаление крайних пробелов и casefold. Согласно документации unicodedata, NFKC сочетает совместимостное разложение с канонической композицией [11]. Исходное название сохраняется для отображения. Служебные поля заполняются моделями при записи через приложение. Здесь нормализация означает преобразование Unicode-строки, а не изменение реляционной схемы. Основные поля и правила сведены в таблицу 3. casefold обрабатывает регистр по правилам Unicode. Такое сравнение не задаёт отдельный словарный порядок русского языка: сортировка идёт по нормализованным строкам. Введённый фрагмент проходит то же преобразование, что и название. Поэтому результат поиска не зависит от того, набран запрос строчными или прописными русскими буквами.

Таблица 3 — Основные поля модели данных

Сущность и поле	**Представление**	**Правило**
Category.id	Целое	Первичный ключ
Category.name	Строка до 100 символов	Непустое; уникальна нормализованная форма
Category.description	Текст или NULL	Не более 2000 символов
Product.id	Целое	Первичный ключ
Product.name	Строка до 200 символов	Непустое; нормализованная копия для поиска
Product.sku	Строка до 64 символов	Уникальный; латинские буквы, цифры, дефис, подчёркивание
Product.category_id	Целое	Внешний ключ; категория существует; RESTRICT
Product.price_kopecks	Целые копейки	От 0 до 2^63 − 1; при вводе рубли строкой
Product.quantity	Целое количество	От 0 до 2^63 − 1
Product.description	Текст или NULL	Не более 2000 символов
Product.created_at, updated_at	Дата и время	Хранятся в UTC; обновление при изменении

Поля name_normalized используются внутри приложения и не входят в публичный ответ API. Исходное написание названия хранится отдельно для отображения пользователю.

Пустое описание после удаления крайних пробелов сохраняется как NULL. Даты создания и изменения записываются в UTC, но сам тип DateTime в SQLite не содержит часового пояса. При подготовке ответа API к значению добавляется обозначение UTC. В HTML исходная дата передаётся элементу time.

Каждое ограничение таблицы проверяет определённое условие. NOT NULL требует значение обязательного поля. CHECK проверяет тип и допустимый диапазон. UNIQUE запрещает совпадения артикула и нормализованного имени категории. Внешний ключ требует существующую категорию. Например, положительное число в category_id может быть допустимо по типу, но отсутствовать в таблице categories. Проверка диапазона такое значение пропустит, а внешний ключ отклонит. При прямой записи тесты отдельно проверяют эти случаи.

Наибольшее целое число SQLite равно 2^63 − 1. Поле price_kopecks содержит копейки, поэтому максимальная принимаемая цена в рублях составляет 92233720368547758.07. Строка с окончанием .08 уже выходит за этот предел. До обращения к базе схема сравнивает значение с границей и возвращает ошибку ввода. Это предотвращает переполнение при сохранении. Для количества используется тот же верхний предел целого числа. Количество проверяется напрямую, без пересчёта денежных единиц. Аналогичная проверка нужна для идентификаторов в адресе. Сначала FastAPI разбирает целое число и проверяет границы параметра Path, затем обработчик обращается к базе. При слишком большом значении запрос останавливается до поиска записи. Для номера страницы верхний предел дополнительно делится на размер страницы, чтобы вычисление смещения оставалось в допустимом диапазоне.

Хранение цены в копейках сохраняет точность сравнения границ: 49,90 рубля превращаются в 4990, а 50 рублей — в 5000. Фильтр может сравнивать целые значения без округления при каждом запросе. Преобразование выполняется до записи через Decimal; строка с тремя дробными знаками отклоняется. Внутреннее поле price_kopecks не принимается из публичного запроса. Клиент передаёт цену в рублях, а сервер сам получает представление хранения и возвращает цену с двумя знаками после разделителя.

При присваивании названия модели обновляется и его нормализованная копия. После переименования поиск использует новую строку, а на странице показывается исходное написание. Прямой SQL не вызывает Python-

валидаторы. Если таким запросом поменять только название, служебное поле останется прежним и поиск может вернуть неверный результат. Поэтому операции приложения записывают данные через модели. При добавлении импорта нужно будет также выполнять нормализацию и проверять соответствие обеих строк.

2.3 АРХИТЕКТУРА ПРИЛОЖЕНИЯ И ПРОЕКТИРОВАНИЕ ИНТЕРФЕЙСА

Компоненты организованы по назначению: database.py создаёт движок и сессии; models.py описывает хранимые сущности; schemas.py проверяет ввод; catalog.py реализует общие операции; маршруты API и страниц адаптируют запросы и ответы. Шаблоны и статические файлы отвечают за представление. Ответственность компонентов приведена в таблице 4, путь запроса — на рисунке 2.

Таблица 4 — Ответственность компонентов

Компонент	**Назначение**
database.py	Движок, внешний ключ, сессия на запрос
models.py / schemas.py	Таблицы, ограничения и проверка входных данных
catalog.py	Общие операции, транзакции, поиск и фильтры
routers/api.py	JSON-контракт и статусы HTTP
routers/pages.py	HTML-формы, сообщения и перенаправления
templates / static	Представление, оформление и местное время
demo_data.py / seed_demo.py	Воспроизводимые примеры, backup и загрузка

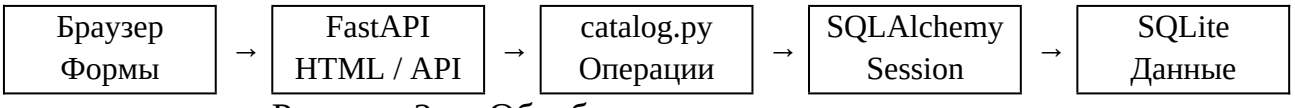


Рисунок 2 — Обработка запроса в приложении

Источник: составлено по модели и исходному коду проекта.

Общие операции вынесены из обработчиков запросов по принципу разделения прикладной логики и кода взаимодействия [5, гл. 4]. В модуле их реализуют функции catalog.py, работающие с сессией SQLAlchemy. Отдельный слой репозиториев, очередь событий и отдельный класс Unit of Work не используются.

Операция записи завершается фиксацией транзакции. При известном нарушении целостности выполняется откат. Требование сохранять связанные изменения целиком рассматривается в описании Unit of Work [5, гл. 6], а состояние сессии после commit и rollback определяет SQLAlchemy [14]. Отдельный класс Unit of Work в проекте не создаётся.

Интерфейс включает список товаров, карточку, формы создания и изменения, подтверждение удаления и страницы категорий. На широком экране меню находится слева, на узком — сверху. Поля формы выстраиваются в один столбец, а широкая таблица прокручивается в своём контейнере. Оформление хранится в локальном CSS.

Рабочая база находится в data/catalog.db. Она не заменяется при обновлении исходников и сохраняется после остановки сервера. В учебных материалах Г. Уилсона рассмотрены сохранение объектов и чтение данных из базы [7]. В проекте для хранения используются таблицы SQLite, а для связи с ними — модели SQLAlchemy.

При конфликте функция каталога возбуждает CatalogError с кодом и описанием. Обработчик API преобразует её в JSON с полем detail, маршрут страницы — в HTML. Шаблоны получают готовые данные и не выполняют операции записи. Поэтому изменение текста сообщения не требует менять поиск дубликата или запрос к базе.

Зависимость get_db создаёт сессию для запроса и закрывает её после использования. Функция каталога выполняет commit, когда запись готова к сохранению. При известном конфликте она делает rollback и возвращает ошибку операции. В тестах зависимость подменяется сессией временной базы. Поэтому тот же обработчик можно вызвать без обращения к рабочему каталогу.

Шаблоны получают уже подготовленные данные и не управляют подключением. Для страниц и API применяется одна зависимость FastAPI.

Для списка и карточки используются разные наборы сведений: в таблице нужны название, категория, цена и остаток, в карточке дополнительно выводятся описание и даты. Форма содержит поля ввода с HTML-ограничениями длины и формата. Серверная схема повторно проверяет запрос, поскольку его можно отправить без браузера.

3 РАЗРАБОТКА И ТЕСТИРОВАНИЕ МОДУЛЯ

3.1 РЕАЛИЗАЦИЯ ОПЕРАЦИЙ С ТОВАРАМИ И КАТЕГОРИЯМИ

Точкой входа служит app/main.py. При запуске обработчик lifespan создаёт отсутствующие таблицы. Путь к базе вычисляется относительно проекта, поэтому не зависит от текущего каталога терминала. Локальный сервер запускается Uvicorn на 127.0.0.1:8000 [20]. Команда create_all сохраняет записи, но не изменяет уже существующую схему.

API содержит десять операций: получение списка, создание, чтение одной записи, изменение и удаление для каждой сущности. Создание товара сначала проверяет схему, наличие категории и уникальность артикула. После добавления объекта выполняется commit, затем обновление его состояния для ответа. Артикул приводится к верхнему регистру. При изменении проверяется существующая запись, новая категория и артикул, после чего заменяются редактируемые поля. Для категорий применяются схемы CategoryWrite и CategoryRead, для товаров — ProductWrite и ProductRead. Входные и выходные поля различаются: клиент передаёт category_id, а в ответе дополнительно получает category_name. Цена в ответе формируется из сохранённых копеек, даты дополняются часовым поясом UTC. Объект модели не возвращается клиенту целиком.

При редактировании сначала проверяются новые значения, затем они присваиваются товару. Если артикул уже занят, функция возвращает отказ до изменения цены и названия. При конфликте во время записи обработчик IntegrityError выполняет rollback и возвращает 409. Порядок восстановления сессии после неудачной записи описан в SQLAlchemy [14]. Тест отправляет конфликтный запрос, сравнивает товар с прежними значениями, а затем выполняет допустимое обновление. Так проверяется продолжение работы после отказа.

Функция delete_category сначала ищет связанный товар. Для проверки достаточно одной найденной записи, поэтому используется LIMIT 1. При пустой категории выполняется удаление и commit. Удаление товара не затрагивает категорию. Поле updated_at меняется средствами модели при выполнении UPDATE.

При ошибке сервер повторно выводит форму с введёнными значениями и сообщениями: для неверного значения используется 422, для известного конфликта — 409. После успеха сервер отправляет 303 на карточку или список. По RFC 9110 этот код указывает на ресурс с результатом исходного запроса [18, п. 15.4.4]. Браузер открывает его методом GET, и обновление страницы не повторяет отправку формы.

HTML передаёт значения строками. Функция integer_fields копирует словарь формы и преобразует category_id и quantity, если они содержат от 1 до 19 десятичных цифр. Затем Pydantic проверяет диапазон [16]. Исходный словарь остаётся доступным для вывода формы, в том числе если строку не удалось преобразовать.

Обработка цены начинается с удаления крайних пробелов и проверки строки регулярным выражением. Допускаются цифры и один десятичный разделитель с одной или двумя последующими цифрами. После замены запятой точкой создаётся Decimal, проверяется предел и вычисляется целое число копеек. Поэтому значения «49», «49.9» и «49,90» приводятся к согласованному формату, а отрицательная цена, экспоненциальная запись и NaN отклоняются. Проверка формата выполняется раньше обращения к базе.

В JSON API поля quantity и category_id принимают только целые числа. Строка «1», число 1.0 и логическое true отклоняются. Для веб-форм действует отдельный этап разбора строк; правила строгой проверки JSON при этом сохраняются.

Схемы входа запрещают лишние поля. Клиент не может задать id, name_normalized или внутреннюю цену в копейках вместе с публичными полями товара. Это проверяется до изменения модели. При PUT обязательные

поля передаются полностью, поэтому запрос только с новым названием отклоняется. Внутри операции редактирования текущий идентификатор исключается из поиска дубликата артикула: сохранение своего прежнего артикула допустимо, совпадение с другой записью — конфликт.

Функция _commit перехватывает IntegrityError и сначала выполняет rollback. Затем по сообщению SQLite определяется известный конфликт: повтор имени категории, занятый артикул или нарушение связи. В этих случаях возвращается ошибка каталога со статусом 409. Если причина не распознана, исключение передаётся дальше. При отладке остаются исходные сведения об ошибке. Такой порядок даёт возможность отдельно проверить ожидаемые отказы ввода и обнаружить неизвестное нарушение ограничения. Для внешнего ключа в _commit можно передать сообщение, подходящее конкретной операции. При удалении категории оно объясняет наличие товаров; при записи товара сообщает об изменении связанной категории. Код конфликта остаётся общим, а причина уточняется по действию. Полный разбор остальных ограничений SQLite эта функция не выполняет.

3.2 РЕАЛИЗАЦИЯ ПОИСКА, ФИЛЬТРАЦИИ И ВЕБ-ИНТЕРФЕЙСА

Отбор выполняет общая функция list_products. Поиск сравнивает нормализованную строку с нормализованным названием товара. Символы процента и подчёркивания экранируются как буквальный ввод для LIKE. К запросу добавляются выбранные ограничения категории, нижней и верхней цены и количества больше нуля. До выборки проверяется корректность диапазона; минимальная цена не может превышать максимальную. Поиск реализован методом contains с autoescape=True. Помимо процента и подчёркивания экранируется служебный символ самого выражения. Пользовательский фрагмент передаётся параметром запроса. Сортировка также не собирается из произвольной SQL-строки: из параметра выбирается заранее заданное выражение.

Параметр sort выбирается из пяти значений: название или цена в обоих направлениях, последнее изменение от новых записей к старым. В list_products каждому значению соответствует выражение SQLAlchemy. Вторым условием всегда задан Product.id по возрастанию. Оно определяет порядок товаров, у которых совпало значение основного поля.

Постраничный вывод возвращает общее число результатов и не более десяти товаров. Сначала считается число записей, удовлетворяющих условиям, затем выбирается нужная часть списка. Обе операции используют одни и те же ограничения. Допустимый запрос с пустой выдачей возвращает успешный ответ.

Jinja2 использует общий базовый шаблон с навигацией и отдельные шаблоны списка, карточки, форм и подтверждений. Пользовательские значения передаются через контекст, как описано в документации FastAPI [13]. Включённое окружением шаблонов автоматическое экранирование обрабатывает HTML-символы; пользовательский текст не помечается safe. Документация Jinja описывает исключения из экранирования и зависимость поведения от настройки [15]. Тесты с введёнными тегами проверяют вывод текста без исполнения разметки.

Для демонстрации большого списка подготовлены 1200 синтетических товаров в 12 категориях. На каждую приходится 100 записей. Отличаются типы, варианты, цены и остатки; 133 товара имеют нулевое количество. Цена находится в диапазоне 19,00–346,10 рубля. Генератор не описывает прайс-лист действующего поставщика. Записи помечены как демонстрационные и имеют стабильные артикулы.

Перед записью в существующую базу загрузчик создаёт резервную копию методом SQLite backup [11]. Метод читает базу через соединение SQLite и учитывает зафиксированные данные. После создания копии начинается транзакция загрузки. Занятые артикулы пропускаются, описания существующих категорий остаются прежними. Повторный запуск сохраняет исправленные менеджером цену и название. Набор имеет постоянные

артикулы, поэтому его можно загрузить на другом компьютере и повторить поиск по тем же позициям. Имена категорий сопоставляются в нормализованном виде. Если категория уже существует, используется её идентификатор и прежнее описание. Перед созданием категорий загрузчик определяет недостающие артикулы. Раздел, все позиции которого уже присутствуют в базе, не создаётся заново. Это важно, если имеющиеся товары ранее были перенесены менеджером в другие категории.

Составной отбор строится как пересечение условий. Например, для запроса «РУЧКА», выбранной категории, диапазона 49,90–100,00 рубля и включённого наличия товар должен удовлетворять всем четырём ограничениям. В тестовом наборе синяя ручка за 49,90 и гелевая за 100,00 попадают в результат, а красная за 99,00 с нулевым остатком исключается. Пример одновременно проверяет регистр поиска, включение граничных цен и независимость наличия от цены.

Для подсчёта и выборки страницы используется один список условий. Сначала считается общее число подходящих строк, затем выполняется запрос с LIMIT 10 и OFFSET. Для первой страницы смещение равно нулю, для второй — десяти. При 13 результатах страницы содержат 10 и 3 записи. Число страниц вычисляется с округлением вверх; даже при пустом результате интерфейс сохраняет первую страницу. Если запрошен номер за пределами выдачи, данные не подменяются результатами другой страницы.

Категория загружается вместе с товаром через joinedload. В строке списка нужно её название, поэтому запрос получает связанные данные до передачи объектов шаблону. Шаблон обращается к product.category.name и не запускает отдельную операцию загрузки для каждой строки. В этом проекте joinedload выбран по составу данных страницы. Сравнение скорости запросов с другими способами загрузки не выполнялось. В автоматическом тесте проверяется, что список содержит правильные названия категорий.

Фильтры передаются методом GET и остаются в адресе. Ссылки страниц строятся через urlencode: действующие параметры сохраняются, меняется

только page. Новая отправка формы начинает поиск с первой страницы. Один и тот же адрес повторяет условия запроса, но состав результата может измениться после редактирования каталога.

3.3 ТЕСТИРОВАНИЕ, АНАЛИЗ РЕЗУЛЬТАТОВ И ОГРАНИЧЕНИЯ РЕШЕНИЯ

Повторная автоматическая проверка выполнена 29.10.2026 в облачной Linux-среде на Python 3.12. Фикстуры создают отдельные временные SQLite-файлы, схему и сессию; тестовый клиент подменяет зависимость get_db. Использование фикстур и освобождение ресурсов описаны в документации pytest [17]. Рабочая база в этих проверках не изменялась.

Всего пройдено 154 теста: 35 проверок базы, 81 API, 31 HTML и 7 загрузчика. Проверяются операции, уникальность, типы и границы чисел, обязательность связи, поиск русского текста, сочетания фильтров, сортировка, пагинация и отказ без частичного изменения. Для загрузчика дополнительно проверены повторное применение, сохранение собственных записей, откат при искусственном сбое и резервная копия с зафиксированными данными WAL. Число 154 включает отдельные наборы параметризованных примеров pytest. Например, одно правило для количества проверяется на нескольких недопустимых типах и значениях. Поэтому количество тестов не равно числу функций, начинающихся с test_. В итоговой таблице группы разделены по файлам базы, API, HTML и загрузчика.

Получено одно предупреждение Starlette об устаревающем использовании HTTPX в TestClient. Оно не привело к отказам тестов. Повторный запуск набора нужен после изменения кода, чтобы обнаружить нарушения уже проверенных сценариев. Эта роль автоматических проверок рассмотрена в Software Engineering at Google [8, гл. 11].

В отчёте от 21.10.2026 зафиксированы 14 сценариев Chromium 154.0.8037.57 на экранах 1440 × 1000 и 390 × 844. Через Playwright проверялись формы, ошибка цены, совместный отбор, переходы страниц, запрет удаления

занятой категории, отмена и подтверждение удаления. Отдельный сценарий выполнялся с отключённым JavaScript. Скрипт использует синхронный интерфейс Playwright для Python [19].

В браузерном сценарии ширина документа сравнивается с шириной окна. Текущие стили ограничивают прокрутку таблицы собственным контейнером, а скрытую подпись размещают внутри него. В отчёте узкий экран отмечен как пройденный сценарий. Для разбора отказа нужны условия его воспроизведения и повторная проверка после изменения; такой порядок описан в The Debugging Book [9].

Второй отчёт от 21.10.2026 содержит шесть сценариев большого каталога: число записей API, переход на следующую страницу, последнюю страницу, совместные фильтры, счётчики категорий и ширину мобильного списка. В наборе 1200 товаров, по 100 в каждой категории; при размере страницы 10 получается 120 страниц. Состав проверок приведён в таблице 5. Проверка временной базы 29.10.2026 дала integrity_check = ok, отсутствие ошибок foreign_key_check и ноль новых товаров при повторной загрузке.

Таблица 5 — Фактические результаты проверки

Набор	**Количество**	**Результат**
База данных	35 тестов	Пройдены
JSON API	81 тест	Пройдены
HTML-страницы	31 тест	Пройдены
Загрузчик примеров	7 тестов	Пройдены
Браузер, основные сценарии	14 сценариев	Пройдены в Chromium
Браузер, 1200 товаров	6 сценариев	Пройдены в Chromium
SQLite большого набора	integrity_check, foreign_key_check	ok; ошибок связей нет

Сохранение записей между запусками проверяется повторным открытием файловой базы другим процессом и повторной инициализацией приложения. В

приложении А приведены список (рисунок А.1), форма (рисунок А.2), ошибка цены (рисунок А.3) и запрет удаления категории (рисунок А.4). Это снимки сценариев от 21.10.2026; позднее надписи и значки интерфейса менялись. Условия получения результатов фиксируются отдельно от последующих изменений, как рекомендовано при документировании происхождения данных [4].

Испытания относятся к локальной работе с каталогом. Нагрузка, одновременная запись несколькими менеджерами и работа на PostgreSQL не проверялись. В приложении нет авторизации, журнала изменений и миграций схемы. Перед расширением режима работы нужны соответствующие доработки и новые испытания.

Проверки хранения и проверки API подтверждают разные части одного правила. Тест базы может выполнить прямую запись недопустимого значения и получить отказ ограничения SQLite. Тест API проверяет, что неправильный запрос вернул 422 или 409 и не изменил каталог. Если использовать только API, ограничение базы могло бы отсутствовать и оставаться незамеченным благодаря входной схеме. Если проверять только базу, могли бы сохраниться неверный код ответа и потеря введённых значений формы.

Граничные примеры выбираются по правилам конкретных полей. Для названия проверяются пустая строка, пробелы и превышение допустимой длины; для цены — ноль, лишняя точность, запрещённые представления и предел целого SQLite. Для количества отдельно рассматриваются отрицательное, дробное, логическое и слишком большое значения. Поиск проверяется на русском регистре и буквальных символах процента, подчёркивания и косой черты. Эти случаи связаны с реализацией схем и запроса, а не со случайным набором товаров.

Тест повторного артикула сохраняет прежние значения товара, отправляет конфликтное обновление и заново читает запись. После отказа поля должны совпадать с исходными. Далее отправляется допустимое обновление и проверяется его результат. В другом сценарии меняется category_id товара.

Ответ должен содержать идентификатор и название новой категории. Если прежняя категория стала пустой, её можно удалить. Проверка охватывает изменение существующей связи и сохранение самого товара.

Для загрузчика искусственный сбой вызывается при выполнении INSERT товара. После исключения проверяется отсутствие и товаров, и категорий, добавленных неудачной транзакцией. Отдельный тест подменяет создание резервной копии ошибкой и подтверждает, что загрузка не начинается. В WAL-сценарии запись сначала фиксируется в журнале открытой базы, затем читается из резервной копии. Так проверяются разные отказы: откат незавершённой загрузки, запрет записи без копии и включение уже зафиксированных данных.

В тесте повторной загрузки сначала создаются собственный товар и категория с описанием. Затем добавляется демонстрационный набор, и у одной его позиции изменяются цена и название. После второго запуска загрузчика тест читает эти записи. Пользовательские значения должны сохраниться, а артикулы не должны повториться. Также проверяется число добавленных и пропущенных товаров. Сравнение полей обнаруживает перезапись данных даже в случае, когда общее количество записей осталось правильным.

Если артикул пользовательского товара совпал с артикулом набора, загрузчик пропускает эту позицию. Для проверки заранее создаётся товар DEMO-01-01-01 в другой категории. После загрузки ожидаются 1199 новых товаров и один пропуск. У исходной записи проверяются прежнее название и категория. Загрузчик использует артикул для определения занятой позиции и не пытается установить её происхождение по описанию. Это правило сохраняет пользовательскую запись при повторном наполнении каталога.

После запуска отдельно проверяется готовность обычного сервера командой scripts.check_ready. Проверка открывает главную страницу с перенаправлением, списки и формы, получает локальные CSS и JavaScript, читает health и списки API. Затем по OpenAPI проверяется наличие десяти операций каталога. Ожидание ответа 200 дополняется проверкой содержания: заголовка HTML, формата JSON и необходимых полей списка. Поэтому

случайно запущенный на том же порту другой сервер не удовлетворит всем условиям только благодаря доступности адреса.

Команда scripts.check_ready читает структуру рабочей базы без добавления товаров. Через SQLAlchemy запрашиваются имена таблиц, столбцы товара и внешний ключ категории. Отдельно проверяется включение PRAGMA foreign_keys для соединения. Ответ health сообщает о работе HTTP-процесса и сам не обращается к каталогу. Поэтому проверка схемы нужна дополнительно: запущенный сервер может отвечать на health при неверной структуре базы. Команду можно выполнять после запуска и обновления исходников; рабочие записи она не меняет.

Браузерные отчёты и снимки относятся к версии интерфейса от 21.10.2026. 29.10.2026 повторены автоматические тесты и проверка демонстрационных данных; новый браузерный прогон не выполнялся. После изменения страниц прежний отчёт сохраняет историческое значение, а актуальное поведение требует отдельного запуска сценариев.

ЗАКЛЮЧЕНИЕ

Разработан локальный модуль управления каталогом товаров на Python 3.12, FastAPI и SQLite. Через веб-интерфейс и JSON API можно создавать, просматривать, изменять и удалять товары и категории, искать позиции и применять фильтры. Для демонстрации используется ассортимент канцелярских товаров.

В работе рассмотрены способы ведения каталога и определены требования к данным. Реализация использует Python 3.12, FastAPI и SQLite. Документация PostgreSQL применена при анализе таблиц и ограничений.

Созданы таблицы категорий и товаров со связью один ко многим. Цена хранится в копейках, остаток — целым числом. Схемы запросов и ограничения базы проверяют допустимость данных.

Десять операций API и веб-формы поддерживают ведение каталога. Поиск сочетается с категорией, ценовым диапазоном и наличием. Порядок выдачи однозначен, переходы между страницами сохраняют условия отбора.

Подготовлен набор из 1200 товаров в 12 категориях. Загрузчик добавляет недостающие позиции, предварительно создавая копию существующей базы. Повторная загрузка сохраняет пользовательские изменения. При отдельной проверке временного набора нарушений целостности не обнаружено.

29.10.2026 повторно пройдены 154 автоматических теста. Сохранённые отчёты Chromium содержат 14 сценариев интерфейса и 6 сценариев большого каталога. Их результаты относятся к проверкам от 21.10.2026.

Модуль выполняет поставленные задачи локального ведения каталога. Исходники, инструкция запуска, тесты и демонстрационные данные доступны в репозитории. Ограничения рассмотрены в разделе 3.3.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.201–78. Единая система программной документации. Техническое задание.

2. Московский финансово-юридический университет МФЮА. Методические указания по подготовке и оформлению курсовых работ/проектов. — Москва, 2025. — Текст : электронный.

3. Downey, A. B. Think Python: How to Think Like a Computer Scientist / A. B. Downey. — 3rd ed. — O’Reilly Media, 2024. — ISBN 978-1-098-15543-8. — URL: https://github.com/AllenDowney/ThinkPython/tree/v3 (дата обращения: 21.10.2026). — Текст : электронный.

4. Research Software Engineering with Python: Building Software that Makes Research Possible / D. Irving, K. Hertweck, L. Johnston [et al.]. — Chapman & Hall/CRC Press, 2021. — ISBN 978-0-367-69834-8. — URL: https://github.com/merely-useful/py-rse (дата обращения: 21.10.2026). — Текст : электронный.

5. Percival, H. Architecture Patterns with Python / H. Percival, B. Gregory. — O’Reilly Media, 2020. — ISBN 978-1-492-05220-3. — URL: https://github.com/cosmicpython/book (дата обращения: 21.10.2026). — Текст : электронный.

6. Percival, H. J. W. Test-Driven Development with Python / H. J. W. Percival. — 3rd ed. — O’Reilly Media, 2025. — ISBN 978-1-098-14871-3. — URL: https://github.com/hjwp/Book-TDD-Web-Dev-Python (дата обращения: 21.10.2026). — Текст : электронный.

7. Wilson, G. Software Design by Example: A Tool-Based Introduction with Python / G. Wilson. — URL: https://github.com/gvwilson/sdxpy (дата обращения: 21.10.2026). — Текст : электронный.

8. Winters, T. Software Engineering at Google: Lessons Learned from Programming Over Time / T. Winters, T. Manshreck, H. Wright. — O’Reilly Media, 2020. — ISBN 978-1-492-08279-8. — URL: https://github.com/abseil/abseil.github.io/blob/master/resources/swe-book/html/ch11.html (дата обращения: 21.10.2026). — Текст : электронный.

9. Zeller, A. The Debugging Book / A. Zeller. — 2021. — URL: https://github.com/uds-se/debuggingbook (дата обращения: 21.10.2026). — Текст : электронный.

10. The Fuzzing Book / A. Zeller, R. Gopinath, M. Böhme [et al.]. — 2021. — URL: https://github.com/uds-se/fuzzingbook (дата обращения: 21.10.2026). — Текст : электронный.

11. Python Software Foundation. Python 3.12 Documentation: sqlite3, decimal, unicodedata. — URL: https://github.com/python/cpython/tree/3.12/Doc/library (дата обращения: 21.10.2026). — Текст : электронный.

12. PostgreSQL Global Development Group. PostgreSQL 18 Documentation: Data Definition. — URL: https://github.com/postgres/postgres/blob/REL_18_STABLE/doc/src/sgml/ddl.sgml (дата обращения: 21.10.2026). — Текст : электронный.

13. FastAPI. Templates: официальная документация. — URL: https://github.com/fastapi/fastapi/blob/master/docs/en/docs/advanced/templates.md (дата обращения: 21.10.2026). — Текст : электронный.

14. SQLAlchemy. Session Basics: официальная документация. — URL: https://github.com/sqlalchemy/sqlalchemy/blob/main/doc/build/orm/session_basics.rst (дата обращения: 21.10.2026). — Текст : электронный.

15. Jinja. Template Designer Documentation. — URL: https://github.com/pallets/jinja/blob/main/docs/templates.rst (дата обращения: 21.10.2026). — Текст : электронный.

16. Pydantic. Strict Mode: официальная документация. — URL: https://github.com/pydantic/pydantic/blob/main/docs/concepts/strict_mode.md (дата обращения: 21.10.2026). — Текст : электронный.

17. pytest. How to Use Fixtures: официальная документация. — URL: https://github.com/pytest-dev/pytest/blob/main/doc/en/how-to/fixtures.rst (дата обращения: 21.10.2026). — Текст : электронный.

18. Fielding, R. HTTP Semantics / R. Fielding, M. Nottingham, J. Reschke. — 2022. — RFC 9110. — URL: https://github.com/httpwg/http-core/blob/main/rfc9110.xml (дата обращения: 21.10.2026). — Текст : электронный.

19. Microsoft. Playwright for Python: Getting Started — Library. — URL: https://github.com/microsoft/playwright/blob/main/docs/src/library-python.md (дата обращения: 21.10.2026). — Текст : электронный.

20. Uvicorn. Settings: официальная документация. — URL: https://github.com/encode/uvicorn/blob/main/docs/settings.md (дата обращения: 21.10.2026). — Текст : электронный.

ПРИЛОЖЕНИЕ А. ЭКРАНЫ И РЕЗУЛЬТАТЫ ДЕМОНСТРАЦИИ

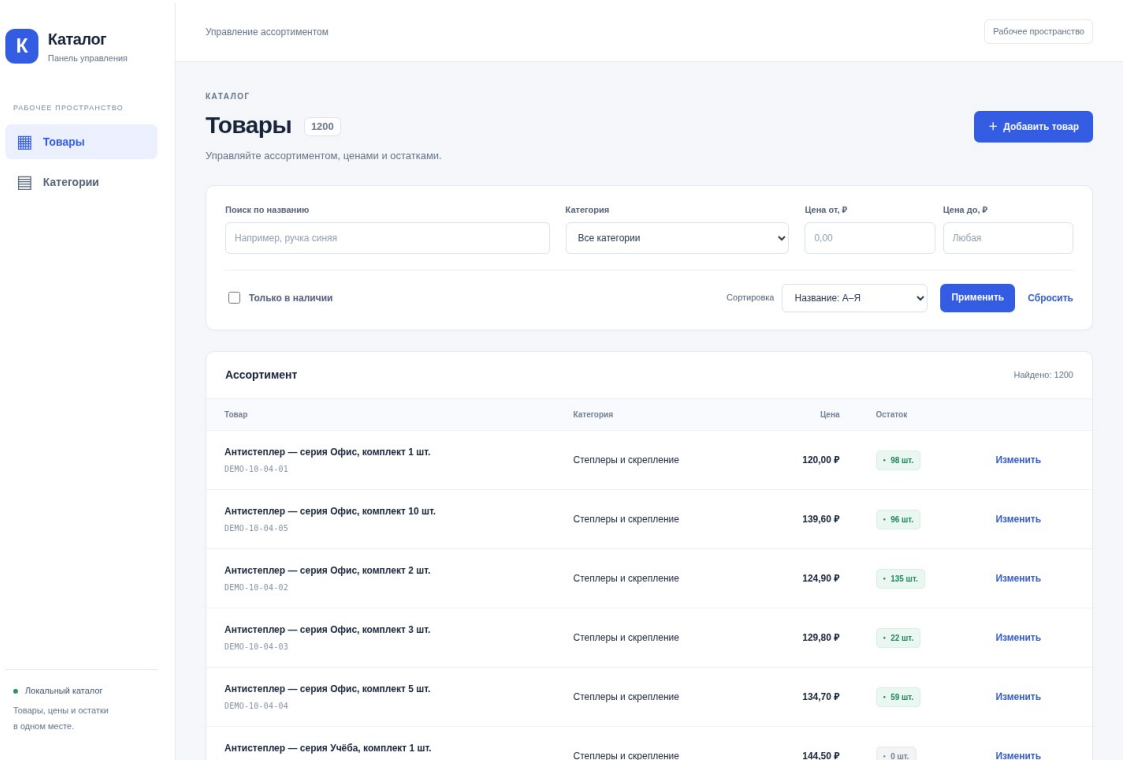


Рисунок А.1 — Каталог из 1200 демонстрационных товаров

Источник: снимок приложения при браузерной проверке 21.10.2026.

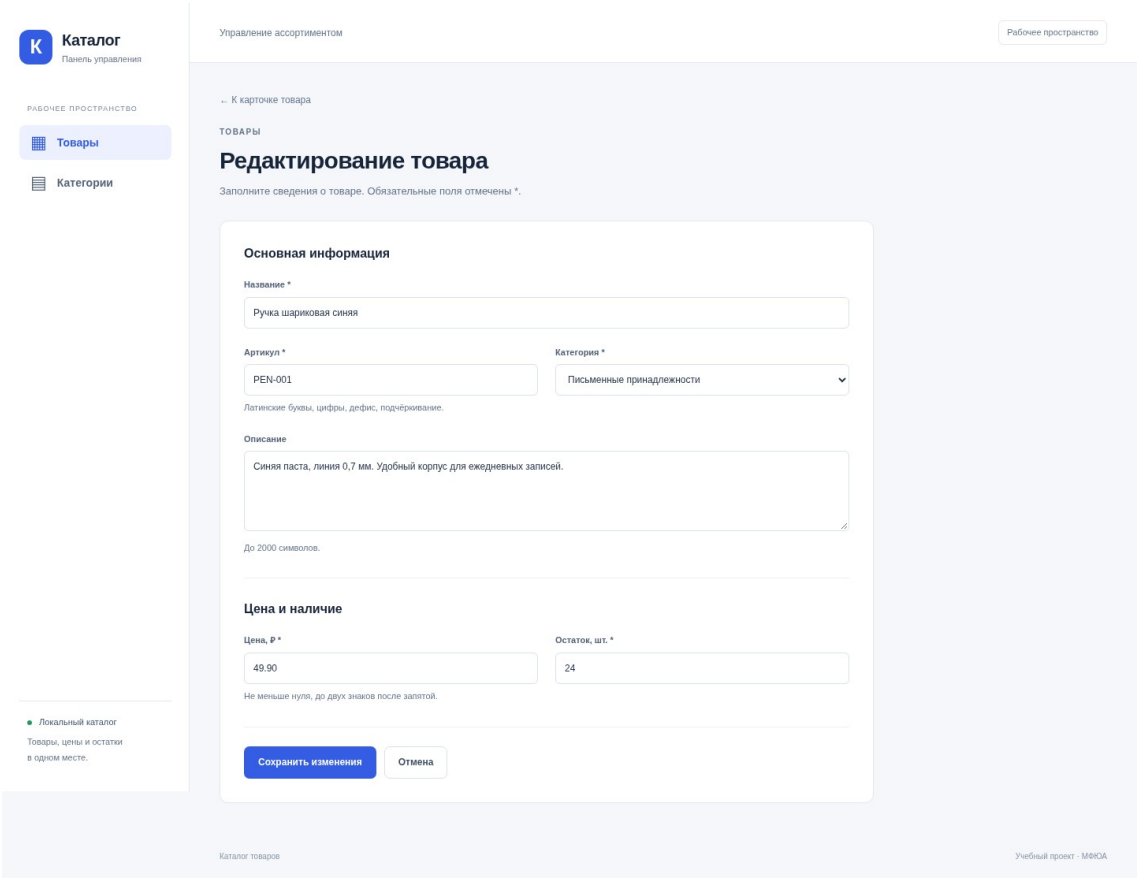


Рисунок А.2 — Форма редактирования товара

Источник: снимок приложения при браузерной проверке 21.10.2026.

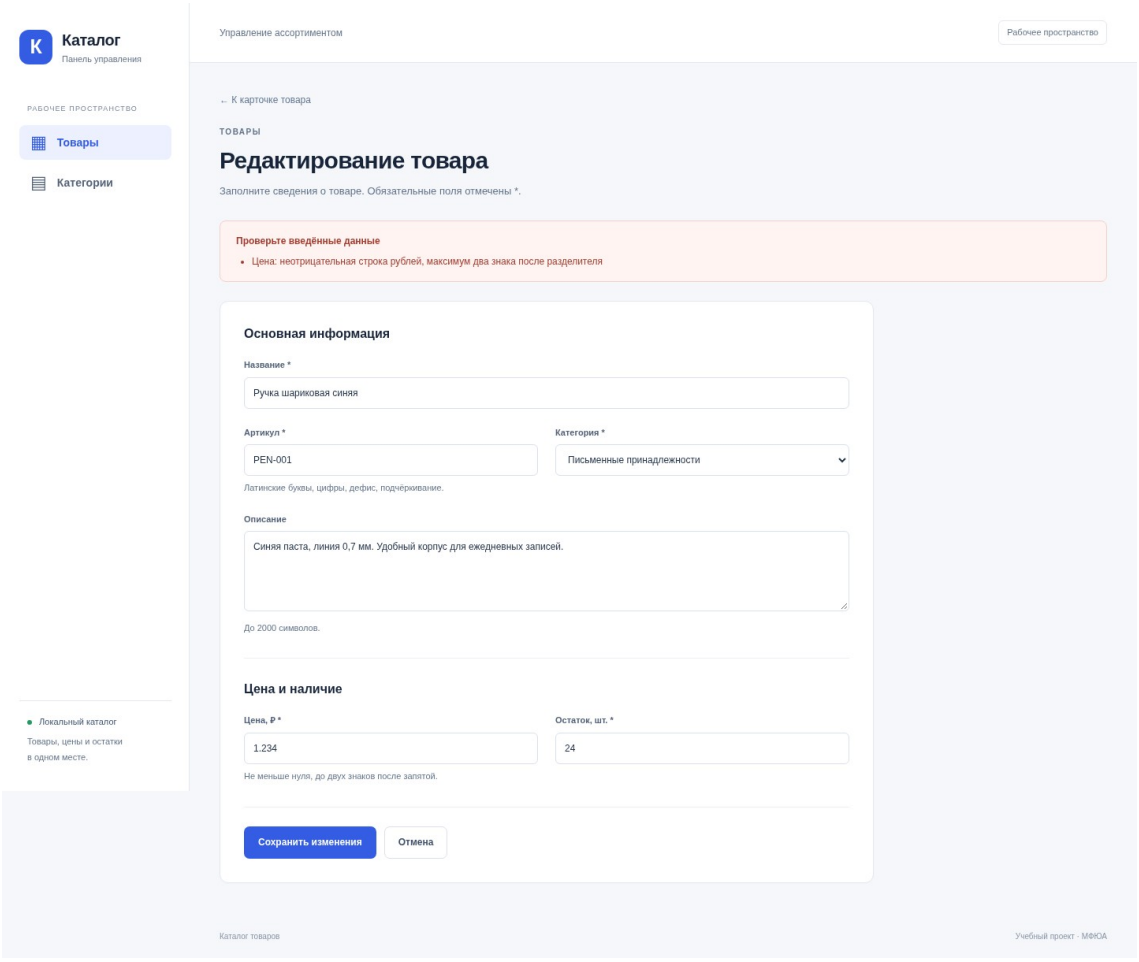


Рисунок А.3 — Сохранение ввода при неправильной цене

Источник: снимок приложения при браузерной проверке 21.10.2026.

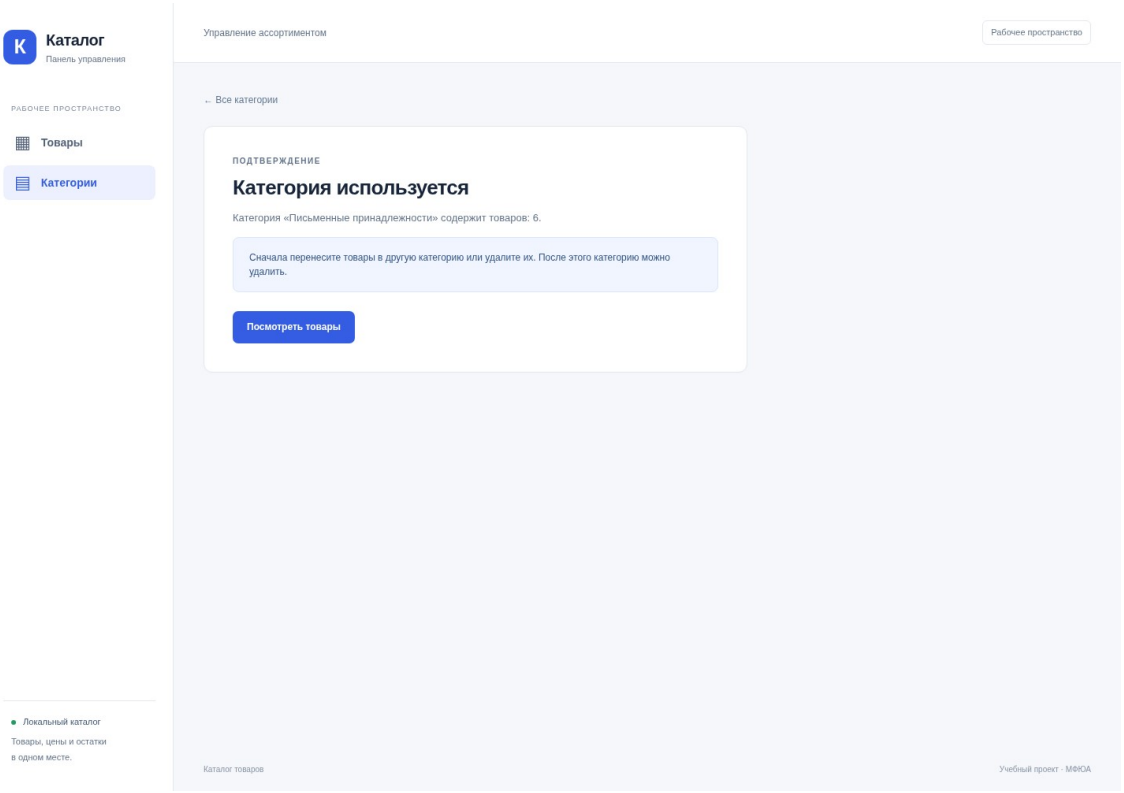


Рисунок А.4 — Запрет удаления категории с товарами

Источник: снимок приложения при браузерной проверке 21.10.2026.